

DevOps Take-Home Assessment

Kubernetes Notes API – Implementation Document

FastAPI application and PostgreSQL database deployed on a local Kubernetes cluster

Component	Implementation
Operating System	Windows 11
Kubernetes Distribution	Docker Desktop Kubernetes
Kubernetes Version	v1.29.1
Application	FastAPI
Database	PostgreSQL 16
Namespace	takehome
Application Exposure	NodePort 30080
Database Exposure	ClusterIP
Database Storage	1 GiB PersistentVolumeClaim

This document records the implementation, validation, persistence test, and troubleshooting performed for the assessment.

1. Objective

The objective was to deploy a Python web application and PostgreSQL database on a lightweight/local Kubernetes environment, connect the application to the database, expose the application externally, and demonstrate persistent database storage.

The implementation uses FastAPI for the application and PostgreSQL for persistence. Kubernetes resources are isolated in the **takehome** namespace.

2. Environment and Platform

The implementation was performed on Windows 11 using Docker Desktop Kubernetes. The final cluster reported a Ready control-plane node running Kubernetes v1.29.1.

Docker Desktop Kubernetes was selected as the compatible local Kubernetes distribution for the laptop environment. The application image was built locally as **notes-api:1.0**.

3. Architecture

The application is exposed through a Kubernetes NodePort. Two FastAPI replicas receive traffic through the application Service. The application connects to PostgreSQL through an internal ClusterIP Service named **postgres**. PostgreSQL stores its data on a 1 GiB PersistentVolumeClaim.

Windows 11
Docker Desktop Kubernetes v1.29.1
Namespace: takehome
NodePort 30080 → FastAPI Service
FastAPI Deployment – 2 replicas
PostgreSQL ClusterIP Service
PostgreSQL Deployment – 1 replica
PersistentVolumeClaim – 1 GiB

4. Application and Containerization

The FastAPI application provides three endpoints: GET /health for health validation, POST /notes for creating a note, and GET /notes for retrieving stored notes. PostgreSQL connectivity is configured through environment variables.

The Dockerfile uses Python 3.12 slim, installs pinned application dependencies, exposes port 8000, and runs Uvicorn. The application container is configured to run as a non-root user (UID 10001).

5. Kubernetes Resources

The deployment is organized into separate Kubernetes manifests for the namespace, Secret, ConfigMap, PersistentVolumeClaim, PostgreSQL Deployment and Service, and application Deployment and Service.

Resource	Purpose
Namespace	Isolates assessment resources in takehome
Secret	Stores PostgreSQL database credentials
ConfigMap	Stores non-sensitive database connection settings
PVC	Provides persistent PostgreSQL storage
PostgreSQL Deployment	Runs the PostgreSQL database
PostgreSQL Service	Provides internal ClusterIP connectivity
FastAPI Deployment	Runs two application replicas
FastAPI Service	Exposes the application using NodePort 30080

6. Configuration, Secrets and Storage

DB_HOST, DB_PORT, and DB_NAME are supplied through a ConfigMap. PostgreSQL database name, username, and password are supplied through a Kubernetes Secret and injected into the workloads using Kubernetes references.

PostgreSQL uses a 1 GiB ReadWriteOnce PersistentVolumeClaim with the hostpath storage class. The database data directory is mounted at /var/lib/postgresql/data.

7. Reliability and Security Features

The application Deployment uses two replicas, a RollingUpdate strategy, HTTP readiness and liveness probes, CPU/memory requests and limits, and a non-root application container. PostgreSQL remains internal to the cluster through ClusterIP.

8. Deployment and Cluster Validation

The Kubernetes node was verified as Ready before validating the workloads.

```
PS C:\devops-assessment-ifs> kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
docker-desktop      Ready    control-plane   58m   v1.29.1
PS C:\devops-assessment-ifs> _
```

Figure 1 – Kubernetes node status: docker-desktop Ready, Kubernetes v1.29.1

```
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                READY   STATUS    RESTARTS   AGE
notes-api-5fff47c8b-bgv4j   1/1     Running    0          44m
notes-api-5fff47c8b-jkkg1   1/1     Running    0          44m
postgres-7d8d5f7878-fxljt   1/1     Running    0          34m
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs>
```

Figure 2 – Two FastAPI replicas and PostgreSQL pod are Running and Ready

9. Services and Persistent Storage

The application Service is exposed as NodePort 30080 while PostgreSQL is available internally through ClusterIP. The PostgreSQL PVC is Bound with 1 GiB capacity and ReadWriteOnce access.

```
PS C:\devops-assessment-ifs> kubectl get svc -n takehome
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
notes-api     NodePort    10.100.66.226 <none>       8000:30080/TCP   47m
postgres      ClusterIP   10.110.246.5  <none>       5432/TCP         47m
PS C:\devops-assessment-ifs>
```

Figure 3 – Application NodePort 30080 and PostgreSQL ClusterIP service

```
PS C:\devops-assessment-ifs> kubectl get pvc -n takehome
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc  Bound     pvc-271e2d62-9a2b-44f5-93b5-2431cd17a289  1Gi        RWO            hostpath       <unset>                 48m
PS C:\devops-assessment-ifs>
```

Figure 4 – PostgreSQL PVC is Bound with 1 GiB capacity

10. Configuration and Secret Validation

The Kubernetes namespace contains the application ConfigMap and database Secret. The screenshot verifies that both resource types exist without exposing the Secret values.

```
PS C:\devops-assessment-ifs> kubectl get configmap,secret -n takehome
NAME                                DATA  AGE
configmap/app-config                3      48m
configmap/kube-root-ca.crt          1      49m

NAME                                TYPE    DATA  AGE
secret/db-secret                    Opaque  3      48m
PS C:\devops-assessment-ifs>
```

Figure 5 – ConfigMap and Secret resources in the takehome namespace

11. API Validation

The application was validated through the externally exposed NodePort. The health endpoint returned the expected healthy status, a note was created successfully, and notes were retrieved from PostgreSQL.

```
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> curl.exe http://localhost:30080/health
{"status":"healthy"}
PS C:\devops-assessment-ifs>
```

Figure 6 – GET /health returns {"status":"healthy"}

```
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> $body = @{ message = "Hello Kubernetes" } | ConvertTo-Json
PS C:\devops-assessment-ifs> Invoke-RestMethod -Uri "http://localhost:30080/notes" -Method Post -ContentType "application/json" -Body $body

id message
--
3 Hello Kubernetes
```

Figure 7 – POST /notes successfully creates a note

```
PS C:\devops-assessment-ifs> Invoke-RestMethod -Uri "http://localhost:30080/notes" -Method Get
id message
--
1 Hello Kubernetes
2 Hello Kubernetes
3 Hello Kubernetes
```

Figure 8 – GET /notes returns persisted note records

The retrieved records demonstrate that the API is reading the stored notes from PostgreSQL rather than returning a static response.

12. PostgreSQL Persistence Test

Persistence was explicitly tested by deleting the running PostgreSQL pod. Because the pod is managed by the PostgreSQL Deployment, Kubernetes automatically created a replacement pod. The PersistentVolumeClaim remained bound to the same persistent volume.

```
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-5fff47c8b-bgv4j           1/1     Running   0           8m22s
notes-api-5fff47c8b-jkkg1           1/1     Running   0           8m12s
postgres-7d8d5f7878-c2cd4           1/1     Running   0           11m
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> kubectl get pvc -n takehome
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc   Bound    pvc-271e2d62-9a2b-44f5-93b5-2431cd17a289   1Gi        RWO            hostpath       <unset>                 11m
PS C:\devops-assessment-ifs>
```

Figure 9 – PostgreSQL pod is deleted and Kubernetes begins recreating it

```
PS C:\devops-assessment-ifs> kubectl delete pod -n takehome -l app=postgres
pod "postgres-7d8d5f7878-c2cd4" deleted
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> kubectl get pods -n takehome -w
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-5fff47c8b-bgv4j           1/1     Running   0           0m56s
notes-api-5fff47c8b-jkkg1           1/1     Running   0           0m46s
postgres-7d8d5f7878-fxljt           1/1     Running   0           14s
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> Invoke-RestMethod -Uri "http://localhost:30080/notes" -Method Get

id message
--
1 Hello Kubernetes
2 Hello Kubernetes
PS C:\devops-assessment-ifs>
```

Figure 10 – Replacement PostgreSQL pod reaches Running state and existing notes remain available

13. Final Validation

Final validation confirmed the Kubernetes node, both application replicas, PostgreSQL, NodePort/ClusterIP services, and the persistent volume were in the expected state.

```
PS C:\devops-assessment-ifs> kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
docker-desktop      Ready     control-plane  27m   v1.29.1
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-5fff47c8b-bgv4j           1/1     Running   0           11m
notes-api-5fff47c8b-jkkg1           1/1     Running   0           11m
postgres-7d8d5f7878-fxljt           1/1     Running   0           114s
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> kubectl get svc -n takehome
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
notes-api           NodePort    10.100.66.226 <none>         8000:30080/TCP   14m
postgres            ClusterIP   10.110.246.5  <none>         5432/TCP         14m
PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> kubectl get pvc -n takehome
NAME                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS    VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc        Bound     pvc-271e2d62-9a2b-44f5-93b5-2431cd17a289  1Gi        RWO            hostpath        <unset>                 14m
PS C:\devops-assessment-ifs>
```

Figure 11 – Final validation of cluster, pods, services and persistent storage

14. Troubleshooting

During the initial deployment, the application pods entered CrashLoopBackOff while PostgreSQL was still initializing. PostgreSQL logs were inspected and confirmed that the database eventually became ready to accept connections. The application Deployment was then restarted using `kubectl rollout restart deployment/notes-api -n takehome`. After the restart, both application replicas became Running and Ready.

A second issue occurred during API testing because curl-style JSON quoting behaved differently in PowerShell. The POST request was changed to PowerShell's `Invoke-RestMethod` with `ConvertTo-Json`, after which note creation succeeded.

15. Conclusion

The FastAPI notes application and PostgreSQL database were successfully deployed on Docker Desktop Kubernetes v1.29.1. The application runs with two replicas and is externally accessible through NodePort 30080. PostgreSQL is exposed internally through ClusterIP and uses a 1 GiB PersistentVolumeClaim.

API validation confirmed successful health checks, note creation, and note retrieval. The PostgreSQL pod recreation test confirmed that previously created records remained available after the database pod was recreated, demonstrating persistent storage.

Submission evidence: This document contains the captured implementation screenshots and the final validation results from the local Kubernetes environment.

Additional Troubleshooting Evidence

The following screenshot provides direct evidence of the application recovery during troubleshooting. The initial state shows both FastAPI pods in **CrashLoopBackOff** with PostgreSQL already Running. The subsequent command output shows both application pods reaching **1/1 Running** after the recovery action.

This demonstrates that the application startup issue was investigated and resolved rather than simply omitted from the implementation record.

```
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                                READY   STATUS             RESTARTS   AGE
notes-api-65685c5db6-fh7xv         0/1     CrashLoopBackOff   3 (40s ago) 101s
notes-api-65685c5db6-hvr5g         0/1     CrashLoopBackOff   3 (43s ago) 101s
postgres-7d8d5f7878-c2cd4         1/1     Running            0           101s
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                                READY   STATUS             RESTARTS   AGE
notes-api-65685c5db6-fh7xv         0/1     CrashLoopBackOff   3 (48s ago) 109s
notes-api-65685c5db6-hvr5g         0/1     CrashLoopBackOff   3 (51s ago) 109s
postgres-7d8d5f7878-c2cd4         1/1     Running            0           109s
PS C:\devops-assessment-ifs> kubectl get pvc -n takehome
NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc                        Bound     pvc-271e2d62-9a2b-44f5-93b5-2431cd17a289 1Gi         RWO             hostpath        <unset>                  2m
PS C:\devops-assessment-ifs> kubectl get pods -n takehome
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-65685c5db6-fh7xv         1/1     Running   4 (68s ago) 2m9s
notes-api-65685c5db6-hvr5g         1/1     Running   4 (71s ago) 2m9s
postgres-7d8d5f7878-c2cd4         1/1     Running   0           2m9s
```

Figure 12 – Troubleshooting evidence: FastAPI pods initially in CrashLoopBackOff, followed by successful recovery to 1/1 Running; PostgreSQL remains Running and the PVC is Bound.

Additional API Validation Evidence

This screenshot provides direct evidence of successful note creation and retrieval through the exposed NodePort endpoint. A POST request creates a note with the message **Hello Kubernetes**, followed by a GET request returning the stored records.

```
PS C:\devops-assessment-ifs> curl.exe -X POST http://localhost:30080/notes -H "Content-Type: application/json" --data-binary "@body.json"
{"id":1,"message":"Hello Kubernetes"}
PS C:\devops-assessment-ifs> $body = @{
>>     message = "Hello Kubernetes"
>> } | ConvertTo-Json
PS C:\devops-assessment-ifs> Invoke-RestMethod -Uri "http://localhost:30080/notes" -Method Post -ContentType "application/json" -Body $body
id message
--
2 Hello Kubernetes

PS C:\devops-assessment-ifs>
PS C:\devops-assessment-ifs> Invoke-RestMethod -Uri "http://localhost:30080/notes" -Method Get
id message
--
1 Hello Kubernetes
2 Hello Kubernetes
```

Figure 13 – API validation: POST /notes successfully creates a note and GET /notes returns the stored records.